

スキルシート

同じ内容を 2 つの形式で公開 — [Markdown](#) (GitHub 原本) ・ [Web](#) (目次つき)

基本情報

- 拠点: 東京都江戸川区
- 現在のポジション: [株式会社ZENSHIN](#) CTO
- エンジニア歴: 10 年目 (2017年～)
- 外部リンク:
 - GitHub: <https://github.com/sugurutakahashi-1234>
 - X: https://x.com/suguru_takaha4
 - Qiita: <https://qiita.com/sugurutakahashi12345>
 - Zenn: <https://zenn.dev/ikuraikura>

強み

- 創業フェーズの CTO 経験
 - 技術選定から開発・運用・社内インフラ整備まで、事業に必要なものを自分で決めて実装
 - サービス 3 つとマッチング 2 サイトを、企画から公開・運用まで担当
 - 小規模でも維持できる構成の選定 (Cloudflare 中心に寄せ、運用コストを抑制)
- AI システム開発
 - LLM で人材マッチングと案件評価を自動化するシステムを設計から運用まで開発
 - RAG・ベクトル検索・AI エージェントを組み込んだアプリケーションを Cloudflare 上に構築
 - 非エンジニアが Claude Code / Codex で操作できるよう、社内システムを MCP 前提で設計
 - AI の判定と人間の判断を記録して定期実行で点検し、プロンプトを更新するループの構築
- FDE としての AI 活用支援
 - 経営層から実装担当者まで、相手に合わせた AI 活用の相談・提案
 - 案件獲得の営業・商談に加え、顧客の業務に合わせた AI ツールの実装・導入も担当
- 0 → 1 リード経験 (受託 4 件 + CTO 1 社)
 - アーキテクチャ選定・CI/CD 構築・チーム運用の設計など、立ち上げの技術面を担当
 - 創業直後のスタートアップでは、1 人目のエンジニアとしてプロダクトと社内システムを構築
 - 受託の内訳: iOS リード 3 件、Flutter リード 1 件
- フルスタックの実装力
 - Swift・React Native・Flutter で iOS / Android、Next.js・Astro で Web を実案件で実装
 - Bun・Hono・oRPC・Drizzle・TanStack Start を本番で運用
 - マルチクラウド (GCP / Cloudflare / AWS) の構成を Terraform で IaC 化し、運用まで担当
- ビジネス KPI 起点での開発経験
 - 獲得コストと LTV を起点に、課金層と無課金層それぞれの収益施策を設計・実装
 - 施策立案から 1 週間でリリースする PDCA を回し、効果が出ない施策の削除も判断
- OSS 開発・技術発信
 - Clean Architecture を基にした設計手法 [Framework-Independent Architecture](#) を考案・公開
 - [ai-chat-md-export](#) など TypeScript 製 CLI ツール 4 本を npm / Homebrew で配布
 - [Zenn](#) と [ZENSHIN の技術ブログ](#) で、AI 活用やプロダクト開発の知見を記事・スライドで発信

職務経歴

フリーランス (2021年7月～現在)

- [No.11] [株式会社ZENSHIN](#) (IT コンサルティング / Sler / SES)
 - CTO / Cloudflare ・ RAG ・ MCP / 自社プロダクト開発と FDE としての顧客支援
- [No.10] ショートドラマアプリ開発 (エンタメ業界 G 社)
 - フロントエンドエンジニア / React Native (Expo) ・ Next.js / モバイル・Web 開発と KPI 施策

- [No.9] NFT ゲームアプリ開発 (Web3 特化 Sler F社)
 - Flutter リードエンジニア / Flutter ・ Riverpod / 0 → 1 立ち上げ
- [No.8] SNS アプリ開発 (Sler E社)
 - iOS リードエンジニア / Swift ・ SwiftUI / 0 → 1 立ち上げ
- [No.7] マーケティングリサーチアプリ開発 (スタートアップ D社)
 - iOS リードエンジニア / Swift ・ SwiftUI ・ Amplify / toC ・ toB 2 サービスの 0 → 1 立ち上げ
- [No.6] ファンクラブアプリ開発 (メガベンチャー C社)
 - iOS エンジニア / Swift ・ UIKit ・ Redux / スタンプラリー ・ 景品交換機能の開発
- [No.5] ドローン制御アプリ開発 (ドローンベンチャー B社)
 - iOS エンジニア / Swift ・ SwiftUI ・ VIPER / 飛行の安定性改善と複数メーカーのドローン対応

外資系ITコンサル A社 (2017年4月～2021年6月)

- [No.4] ショッピングアプリ開発
 - Flutter エンジニア / Flutter ・ Google Maps API / iOS ・ Android 両対応のデモアプリ開発
- [No.3] 飲食店管理アプリ開発
 - iOS リードエンジニア / Swift ・ SwiftUI ・ MVVM / BtoB 向け MVP 開発
- [No.2] クレジットカードアプリ iOS 開発
 - iOS エンジニア / Swift ・ UIKit ・ MVVM / サーバーサイドと兼任での画面実装 ・ レビュー
- [No.1] クレジットカードアプリ API 開発
 - サーバーサイドエンジニア / PHP (CodeIgniter) ・ Java / リニューアルに伴う API ・ バッチ開発

案件詳細

[No.11] 株式会社ZENSHIN (IT コンサルティング / Sler / SES)

チーム体制

- CTO として、顧客企業の AI 活用支援と社内プロダクト ・ インフラの開発 ・ 運用を一人で担当

案件概要 ・ 担当業務

- 創業フェーズの株式会社ZENSHIN の CTO として、以下 6 つの業務を並行して推進
 - **[業務1] 自社サービスの企画 ・ 立ち上げ**: 3 サービスとマッチング 2 サイトの公開
 - **[業務2] AI 活用支援 / 提案活動 (FDE)**: 商談 ・ 要件定義から実装 ・ 導入まで担当
 - **[業務3] AI マッチングシステム開発**: Cloudflare 上の RAG / MCP / AI エージェント基盤
 - **[業務4] AI 案件選別システム開発**: 外部案件を LLM で自動評価する社内システム
 - **[業務5] [ホームページ](#) ・ [技術ブログ](#)**: 設計 ・ 構築 ・ 運用
 - **[業務6] 社内インフラ管理**: Terraform によるマルチクラウド IaC

経験した技術

- **FDE / AI 活用支援 (業務2)**
 - Claude Design や v0 による商談前の高速プロトタイプング
 - LLM ツールの使い分けの提案、プロンプト改善、MCP 化 ・ Skills 設計の方針策定
 - 建設業向けに、過去事例 ・ 基準類 ・ 図面から施工計画書の下書きを生成する仕組みを開発
- **RAG / ベクトル検索マッチング (業務3)**
 - Workers AI (bge-m3) + Vectorize による意味ベースの類似検索
 - ベクトル検索 (一次絞り込み) → AI エージェント採点 (二次精査) の 2 段パイプライン設計
 - 自作の評価スクリプトで LLM を比較検証し、シーンごとに使い分け
- **MCP サーバー / AI エージェント運用 (業務3)**
 - 70 以上のツールを持つ MCP サーバーの開発 (OAuth 認証 ・ マルチテナント認可)
 - WebMCP (ブラウザ内 MCP) の Origin Trial 先行導入
 - 定期実行と 50 以上のスキルによる業務自動化 (Claude Code / Codex 両対応)
- **LINE 公式アカウント基盤 (業務3)**
 - 友だち紐付け不要の本人アカウント連携

- スタッフ向けの双方向チャット
 - LINE の Quick Reply（選択式の返信）による稼働状況の定期ヒアリング
- **Cloudflare フルスタック（業務1・3・4・5）**
 - Workers / Workflows / Queues / D1 / Vectorize / Browser Rendering によるサーバーレス構成
 - Hono + oRPC + React 19 / TanStack Start によるマッチング 2 サイトと社内 Web アプリ
 - Astro によるホームページと技術ブログ
 - Email Sending・Slack Interactivity による採用応募対応の自動化
- **マルチクラウド IaC（業務6）**
 - GCP / Cloudflare / AWS / Google Workspace を Terraform で横断管理
 - Workload Identity Federation (OIDC) による鍵レス認証

取り組み・貢献

- **自社サービスの立ち上げ（業務1）**
 - AI 活用伴走・業務変革コンサルティング・スポット開発の 3 サービスを企画
 - [ZENSHIN フリーランス](#)（BtoC）と [ZENSHIN エンジニアマッチ](#)（BtoB）を公開
- **顧客の業務と体制に合わせた提案・納品（業務2）**
 - 決裁者と実際に使う担当者の双方にヒアリングと業務観察を行い、要件を固めて受注
 - 顧客が契約済みの AI（ChatGPT・Gemini など）と利用状況に合わせて構成を選定
- **営業による AI エージェントの実務運用（業務3）**
 - IT 知識のない営業が Claude Code への自然言語指示で要員検索から人材紹介まで完結
 - 案件 1,000 件 × エンジニア 1,000 名の規模から、5 分以内に適切なマッチングを提示
- **AI の判定が顧客接点まで届く自動ループの構築（業務3・4）**
 - 案件収集 → AI 採点 → 本人 LINE への自動提案 → 回答受領まで、人手を介さない運用を実現
 - AI エージェントの返信・提案には人間の承認と監査記録を必須化
- **AI を過信しない設計・運用（業務3・4）**
 - 条件ミスマッチにはサーバー側で決定論的にスコア上限を強制し、AI の過大評価を防止
 - AI の判定と人間の判断のずれを本番データで分析し、プロンプトを 25 回以上改修
 - 構造化のような変換は決定論的な処理に寄せ、AI の担当範囲を限定
- **社外への技術発信（業務5）**
 - 技術ブログを Astro で立ち上げ、AI 活用やプロダクト開発の知見を記事とスライドで公開
 - SEO 施策（構造化データ・sitemap・llms.txt）を入れ、Lighthouse スコア 100 を維持
- **運用を継続できる仕組みの整備（業務6）**
 - 社内インフラを IaC 化し、コード変更 → PR → plan CI → apply のフローを確立
 - 採用しなかった選択肢を含め、技術的な意思決定の根拠をリポジトリに記録する文化を形成

開発環境

- **言語 / ランタイム**
 - TypeScript , Bun , Python (uv)
- **フロントエンド**
 - Astro , React 19 , TanStack Start / Router / Query , shadcn/ui , Tailwind CSS v4
- **バックエンド / AI システム**
 - フレームワーク: Hono , oRPC , Drizzle ORM , Zod , Better Auth
 - MCP: @modelcontextprotocol/sdk v2 , WebMCP
 - 外部 API 連携: LINE Messaging API , Slack API , Google Drive API , GitHub App
- **クラウド / インフラ**
 - Cloudflare: Workers AI , AI Gateway , Vectorize , Workflows , Queues , Browser Rendering , Email Sending
 - マルチクラウド / IaC: GCP , AWS , Terraform , tfllint , dotenvx , mise
- **CI/CD**
 - GitHub Actions , release-please , Wrangler , Renovate
- **テスト / 品質**

- Vitest , Playwright , Storybook , MSW , @axe-core/playwright , oxlint , oxfmt , knip , lefthook
- AI ツール (提案活動・開発)
 - Claude Code , Codex , v0 , Chrome DevTools MCP
- 分析 / エディタ
 - 分析: Google Search Console , PageSpeed Insights , PostHog , Cloudflare Analytics
 - エディタ: Visual Studio Code

[No.10] ショートドラマアプリ開発 (エンタメ業界 G社)

チーム体制

- 案件全体人数 : 6名
 - フロントエンドエンジニア (iOS + Android + Web) : 2名 (担当)

案件概要・担当業務

- 0 → 1 直後のグロースフェーズにおけるショートドラマアプリの開発
- React Native (Expo) で iOS / Android を同時開発し、Next.js で Web 版の開発も担当

経験した技術

- **React Native (Expo) / Next.js**
 - Monorepo 構成による iOS / Android / Web のクロスプラットフォーム開発
 - Expo SDK のメジャーバージョンアップ対応 (v52 → v53 → v54 → v55)
 - Solito によるモバイル (Expo) と Web (Next.js) 間のコード共有
 - Tamagui を用いたプラットフォーム間の UI 統一
 - expo-iap によるアプリ内課金の実装
 - Rive を用いたアニメーションの組み込み
 - (この行を削除)
 - TanStack Query によるデータ取得とキャッシュ管理
 - Crisp SDK を用いたカスタマーサポート用チャット機能の実装
- **開発プロセス改善**
 - Storybook による UI コンポーネントカタログの作成
 - Maestro によるモバイルアプリでの E2E テストの導入
 - mise による開発環境セットアップの簡略化とツールバージョンの統一
- **AI・MCP 活用**
 - Figma MCP を用いたデザインからの実装
 - Storybook で作成したコンポーネントを Claude Code が Chrome DevTools MCP / Playwright MCP で検証
 - GitHub Actions による Claude Code 自動レビューの仕組みの構築

取り組み・貢献

- **獲得コストと LTV を起点としたグロース施策**
 - 獲得コストに課金 LTV が届かない前提を CEO と共有し、LTV を上げる施策と一緒に設計
 - CEO の数値分析から施策の意図を読み取り、アプリの画面と導線に反映
 - 課金導線・ガチャ・ミッション・無料開放を実装し、サブスク契約者数と ARPU を改善
 - 課金転換しない層からはリワード広告・オフアーウォールで収益化
- **週次リリースでの仮説検証**
 - 施策立案からリリースまでを 1 週間で回し、スプリントごとに反復
 - 季節要因や曜日パターンの影響を切り分け、A/B テストで施策単体の効果を測定
 - 効果が出なかった施策はすぐに削除し、その結果も次の意思決定に反映
- **デザイナーとの協業**
 - 表示内容や画面遷移、コンポーネントの色や配置まで踏み込んだ UI 仕様の改善提案
 - 仮実装を早い段階で共有し、議論しながらリリース前に改善
 - リリースごとの KPI をデザイナーに共有し、次の改善対象の優先順位を決定

開発環境

- **React Native (Expo) / Next.js**
 - アーキテクチャ: Monorepo
 - 主要ライブラリ: TanStack Query , ts-proto , Tamagui , Solito , Expo Modules API
 - コード品質・テスト: Storybook , ESLint , Prettier , Vitest , Maestro
- **Firebase**
 - Authentication , App Distribution , Remote Config , Analytics
- **Google Cloud**
 - Cloud Run
- **分析ツール**
 - BytePlus DI , Looker Studio , Adjust
- **CI/CD**
 - GitHub Actions
- **プロジェクト管理**
 - GitHub Projects , Notion , Linear
- **開発ツール**
 - VSCode , Android Studio , Xcode , GitHub Copilot , Claude Code , Codex , Gemini
- **デザインツール**
 - Figma

[No.9] NFT ゲームアプリ開発 (Web3 特化 Sler F社)

チーム体制

- 案件全体人数 : 6名
 - PdM : 1名
 - PM : 1名
 - デザイナー : 1名
 - サーバーサイドエンジニア : 2名
 - Flutter エンジニア : 1名 (担当)

案件概要・担当業務

- 位置情報を使った NFT ゲームアプリの 0 → 1 立ち上げ案件
- アーキテクチャ考案・ライブラリ選定から実装までを担当
- PM・デザイナー・サーバーサイドチームとの仕様調整も担当

経験した技術

- **Flutter**
 - Riverpod と Hooks を用いた状態管理
 - Google Maps API と google_maps_flutter ・ geolocator による地図表示と位置情報の取得
 - go_router を用いた画面遷移の実装
 - openapi_generator を用いた API クライアントコードの自動生成
 - dio を用いた HTTP 通信と、インターセプターによる JWT 認証の実装
 - flutter_secure_storage を用いた機密データの保存
 - permission_handler を用いた位置情報・カメラ・写真フォルダの権限取得
 - slang による多言語対応
 - pedantic_mono による静的解析ルールの統一
 - ThemeData によるデザインシステムの実装
 - fvm による Flutter のバージョン管理
- **開発プロセス改善**
 - Prism を活用した API モックサーバーの構築
 - [Lefthook](#) による pre-commit フックでの静的解析の実行

取り組み・貢献

- 開発体制の改善

- GitHub Projects のスクラムボードで、タスクの進捗を可視化
 - デイリーミーティングで、メンバー間の情報共有と開発プロセスの改善を推進
 - バグの発見から修正までのプロセスを整備し、チーム内で運用
- **API インターフェース設計と UI 先行開発**
 - サーバーサイドの PR をレビューし、API インターフェースの改善点をフィードバック
 - OpenAPI 定義から Prism でモックサーバーを整備し、UI の先行開発を実施
- **デザインシステムの導入**
 - デザイナーと共同でデザインシステムを設計し、画面間の一貫性を確立
 - デザインシステムを Flutter の `ThemeData` で定義し、UI の実装コードを削減

開発環境

- **Flutter**
 - アーキテクチャ: `Riverpod` , `Hooks`
 - 主要ライブラリ: `go_router` , `dio` , `slang` , `permission_handler` , `flutter_secure_storage` , `freezed` , `google_maps_flutter` , `geolocator`
 - コード品質・開発支援: `pedantic_mono` , `openapi_generator` , `fvm` , `Prism` , `Lefthook`
- **CI/CD**
 - `GitHub Actions`
- **プロジェクト管理**
 - `GitHub Projects` , `Notion` , `Slack`
- **開発ツール**
 - `VSCode` , `Android Studio` , `Xcode` , `GitHub Copilot` , `ChatGPT`
- **デザインツール**
 - `Figma`

[No.8] SNS アプリ開発（Sler E社）

チーム体制

- 案件全体人数：約10名
 - iOS エンジニア：1名（担当）

案件概要・担当業務

- 0 → 1 フェーズでの SNS アプリの立ち上げ案件
- 唯一の iOS エンジニアとして、アーキテクチャ・ライブラリ選定・CI/CD 構築から実装まで担当
- PM・デザイナー・サーバーサイドチームとの仕様調整も担当

経験した技術

- **Swift**
 - Xcode 16 Beta での Swift 6 対応（Strict Concurrency を含む）
 - AVFoundation を活用した録音・再生機能の実装
 - [WhisperKit](#), [Speech](#) SDK を活用した音声データの文字起こしの実装
- **開発プロセス改善**
 - [Swift OpenAPI Generator](#) で API 通信処理を自動生成する GitHub Actions パイプライン
 - [Swagger UI Action](#) で API 仕様書を自動生成する GitHub Actions パイプラインの構築
 - [tbls](#) で MySQL のテーブル定義書を自動生成する GitHub Actions パイプラインの構築
 - [pixelmatch](#) による View のスナップショット差分検出の実装

取り組み・貢献

- **アジャイル開発**
 - テスタブルなアーキテクチャの導入: モック化したレスポンスで、API 提供前から実装に着手
 - 検証用デバッグ画面の作成: 新機能や View を早期に確認
 - Docs as Code の導入: [Swagger UI Action](#) や [tbls](#) によるドキュメント生成を調査し、サーバーサイドチームに展開
 - 先行着手: ワイヤフレーム段階から iOS アプリを形にし、仕様と実装の課題をチームへ共有

- **CI/CD 環境の構築**
 - Xcode Cloud 導入: PR マージをトリガーに TestFlight 配信を自動化し、新機能を迅速に検証
 - API インターフェース変更への追従: OpenAPI の差分をトリガーに、iOS リポジトリへ Pull Request を自動作成する GitHub Actions を構築
 - スナップショットテスト: View の差分検出環境を構築し、不具合を早期に発見
- **iOS メンバーの増員や引き継ぎを見越したドキュメント・進捗の整備**
 - ドキュメント整備: 環境構築手順・ライブラリ選定理由・アーキテクチャ・CI/CD 構成図・ブランチ戦略を README に記載
 - プロジェクト管理: リリースノート・タグ・マイルストーン・GitHub Projects を整備し、タスクの進捗を時系列で追跡
- **Swift 6 移行とテストコードの自動生成**
 - コード生成: View 層・UseCase 層のボイラプレートとテストコードを [Sourcery](#) や [Mockolo](#) で自動生成
 - Swift 5 → Swift 6 への移行: Beta 版 Xcode で先行検証し、Strict Concurrency 対応を完了

開発環境

- **Swift**
 - アーキテクチャ: Clean Architecture , Swift Package Manager (マルチモジュール構成)
 - Swift 標準 SDK & API: SwiftUI , Swift Package Manager , Swift Concurrency , Combine , AVFoundation , Speech , Swift Testing , String Catalogs , Swift OpenAPI Generator
 - サードパーティ製 SDK: WhisperKit , Sourcery , Mockolo , pixelmatch
- **CI/CD**
 - Xcode Cloud , GitHub Actions , Renovate
- **プロジェクト管理**
 - GitHub Projects , Notion , Backlog
 - インターフェース共有
 - OpenAPI , Swagger UI
- **デザインツール**
 - Figma

[No.7] マーケティングリサーチアプリ開発 (スタートアップ D社)

チーム体制

- 案件全体人数: 約15名
 - iOS エンジニア: 3名 (リードエンジニア担当)

案件概要・担当業務

- スタートアップ企業の 0 → 1 フェーズでのマーケティングリサーチサービス立ち上げ案件
- BtoC 向けコンテンツ配信アプリと BtoB 向け BI ツールの 2 サービス構成で、iOS チームのリードエンジニアを担当

経験した技術

- **Swift**
 - iOS 16 以上を対象とした SwiftUI での画面開発
 - Clean Architecture × Swift Package Manager でのマルチモジュール構成の構築
 - Xcode Cloud での CI/CD 環境の構築
 - Protocol Buffers に対応した [SwiftProtobuf](#) を用いたデータ連携
 - Swift Concurrency (async/await, AsyncStream, TaskGroup, Actor) を用いた非同期処理のハンドリング
 - [AWS Amplify SDK](#) を用いた Cognito による SMS 認証・認可、AppSync による GraphQL 疎通、Pinpoint によるログイベント送信、S3 とのデータ連携
 - デザインシステムを活用した画面実装
 - AVFoundation を用いた動画の再生
 - ReplayKit を用いた画面の録画
 - Combine を用いた視線や感情の時系列データのハンドリング
 - JavaScript を用いたアプリ内 WebView のイベント処理
- **開発プロセス改善**
 - リリースタグ・リリースノート・レビュー追加を自動化する GitHub Actions Workflow
 - [Renovate](#) によるライブラリ更新 PR の自動作成

- [Periphery](#) による Swift の未使用コードの静的解析
- Swift-DocC によるドメイン層のドキュメント化
- [Mockolo](#) によるテスト用モックの自動生成
- GitHub Copilot, ChatGPT の活用

取り組み・貢献

- 0 → 1 フェーズの iOS リードとして、アーキテクチャ・ライブラリの選定、ブランチ戦略・リリース手順の確立を担当
- CI/CD 環境の構築と、iOS チームのスクラムボード運用の設計
- チームに経験者のいない AWS Amplify SDK や SwiftProtobuf をサンプルアプリで先行検証し、採用まで推進
- 視線分析・感情分析の SDK を、入れ替えても影響範囲が最小になるアーキテクチャで組み込み
- PdM・デザイナー・サーバーサイド・データ分析と、仕様やデータ連携インターフェースを調整
- iOS チームの Issue 運用を担い、メンバーのタスクが途切れない状態を維持

開発環境

- **Swift**
 - アーキテクチャ: VIPER , Clean Architecture , Swift Package Manager
 - Swift 標準 SDK & API: SwiftUI , Swift Package Manager , Swift Concurrency , Combine , Swift-DocC , AVFoundation , Core ML , WebKit , ReplayKit , Logger
 - サードパーティ製 SDK: SwiftProtobuf , Firebase , Amplify , Quick/Nimble , LicensesPlugin , PhoneNumberKit , DeviceKit , SwiftFormat , SwiftGen , Lottie , Mockolo , Mint , Periphery
- **クラウド連携**
 - AWS Amplify: AppSync (GraphQL) , Cognito , S3 , Pinpoint
 - Firebase: Crashlytics
- **CI/CD**
 - Xcode Cloud , GitHub Actions , Renovate
- **プロジェクト管理**
 - GitHub Projects , Notion , Backlog
- **インターフェース共有**
 - Protocol Buffers , Swagger
- **デザインツール**
 - Figma

[No.6] ファンクラブアプリ開発（メガベンチャー C社）

チーム体制

- 案件全体人数：約30名
 - iOS エンジニア：5名（担当）

案件概要・担当業務

- アーティストのファンクラブアプリで、スタンプラリー・景品交換機能を開発
- デザイナーとの仕様調整、見積もり、実装、レビュー、バグ修正を担当

経験した技術

- VueFlux を用いた Redux ベースの状態管理
- デザインシステムに沿った UI の実装

取り組み・貢献

- Android・Web フロントのエンジニアと連携し、プラットフォーム間で仕様差が出ないように調整

開発環境

- **Swift**
 - アーキテクチャ: Redux
 - Swift 標準 SDK & API: UIKit , AVFoundation

- サードパーティ製 SDK: Carbon , VueFlux , ReactiveSwift , XcodeGen , Quick/Nimble , APIKit , CocoaPods , Carthage , Lottie
- クラウド連携
 - Firebase: Crashlytics
- CI/CD
 - CircleCI , Fastlane
- プロジェクト管理
 - Wrike , Kibela
- インターフェース共有
 - Protocol Buffers , Swagger
- デザインツール
 - Figma

[No.5] ドローン制御アプリ開発（ドローンベンチャー B社）

チーム体制

- 案件全体人数：約15名
 - iOS エンジニア：6名（担当）

案件概要・担当業務

- BtoB 向けドローン制御アプリで、飛行の安定性改善・複数メーカー対応・画面開発を担当
- アーキテクチャの検討、見積もり、実装、レビュー、バグ修正を担当

経験した技術

- Swift**
 - アーキテクチャの比較検討と Clean Architecture での実装
 - Clean Architecture での実装
 - SwiftUI・UIKit と Combine を組み合わせた画面実装
 - Swift Concurrency による非同期処理のハンドリング
 - Firebase Crashlytics、Xcode Organizer を用いたバグの原因調査
 - Logger API を用いたログ出力
 - Quick/Nimble を用いたテストコードの記述
 - Quick/Nimble と Mock を用いたテストコードの記述
 - iPad の画面サイズへの対応
- IoT**
 - 外部ライブラリを用いたドローン制御の Swift 実装
 - PID 制御などの制御工学に基づく適切な制御モデルの Swift 実装
 - ROS (Robot Operating System) 環境の活用

取り組み・貢献

- Clean Architecture の採用により、UI を変えずにライブラリを差し替え
- レイヤーごとに独立したテストを実現
- UIKit と Delegate パターンによる実装を SwiftUI・Combine・Swift Concurrency へ置き換え
- チーム運用の改善**
 - 見積会・レトロスペクティブ・開発チーム朝ハドル会の定例化
 - レトロスペクティブの実施
 - 開発チーム朝ハドル会の実施
 - プロダクトバックログに「Ready」（着手できる状態）の定義を導入
 - Pull Request の作成からマージまでの運用ルールの見直し
 - リリースブランチ運用の見直し
 - デイリーミーティング前の Slack リマインダーの設定
 - デイリーミーティングでの相談事項に事前エントリー制を導入
 - Firebase Crashlytics 運用の見直し

開発環境

- **Swift**
 - アーキテクチャ: VIPER , Clean Architecture
 - Swift 標準 SDK & API: SwiftUI , UIKit , Combine , Swift Concurrency , Logger , MetricKit
 - サードパーティ製 SDK: Realm , Quick/Nimble , APIKit , CocoaPods , Carthage
- **クラウド連携**
 - Firebase: Crashlytics , Analytics
- **CI/CD**
 - Bitrise , Fastlane
- **プロジェクト管理**
 - Zenhub
- **デザインツール**
 - Figma

[No.4] ショッピングアプリ開発

チーム体制

- 案件全体人数 : 2名
 - Flutter エンジニア : 1名 (担当)
 - デザイナー : 1名

案件概要・担当業務

- ショッピングアプリのデモを Flutter で開発し、iOS・Android の両方に対応
- Flutter のフロントエンドから Firebase のバックエンドまでを一人で担当

経験した技術

- **Flutter**
 - Provider による状態管理
- **Firebase**
 - Authentication によるユーザー認証
 - Firestore へのデータの永続化
 - Firestore での NoSQL DB 設計
 - Storage への画像データの永続化
 - Crashlytics によるクラッシュレポートの収集・管理
 - App Distribution によるテスターへのアプリ配布
 - Analytics による KPI の計測
 - Google Maps API による地図の表示
- **CI/CD**
 - Codemagic での iOS・Android のアプリ配布の自動化
 - Fastlane を用いた App Distribution へのアップロード

取り組み・貢献

- Firebase によるサーバーレス構成でのモバイルバックエンドの設計
- Flutter での Android 対応 (Material Design、Google Play ストアでの配信)

開発環境

- **Flutter**
 - アーキテクチャ: Provider
- **クラウド連携**
 - Firebase: Authentication , Firestore , Storage , Crashlytics , App Distribution , Analytics
 - Google Maps API
- **CI/CD**

- Codemagic , Fastlane
- デザインツール
 - Adobe XD

[No.3] 飲食店管理アプリ開発

チーム体制

- 案件全体人数 : 約10名
 - iOS エンジニア : 3名 (リードエンジニア担当)

案件概要・担当業務

- BtoB 向け飲食店管理アプリの MVP 開発
- iOS リードエンジニアとして、要件調整とサーバーサイドとの API インターフェース検討を担当
- スクラムマスターも兼任

経験した技術

- Swift
 - SwiftUI での画面実装
 - Codable プロトコルを用いた JSON のエンコード・デコード
 - TestFlight によるアプリ配信
 - アーキテクチャ・ディレクトリ構成の検討
- スクラムマスター
 - スクラムボードの設計
 - スクラムイベントのファシリテーション

取り組み・貢献

- 指定された要件の実装にとどまらず、顧客やデザイナーへ仕様・デザインの改善を提案
- レトロスペクティブを待たず、スクラムボードの運用やコードレビューをチーム内で相談し改善

開発環境

- Swift
 - アーキテクチャ: MVVM
 - Swift 標準 SDK & API: SwiftUI
 - サードパーティ製 SDK: SwiftLint
- プロジェクト管理
 - ZenHub , Trello
- デザインツール
 - Adobe XD

[No.2] クレジットカードアプリ iOS 開発

チーム体制

- 案件全体人数 : 約20名
 - iOS エンジニア : 4〜5名 (担当)

案件概要・担当業務

- BtoC 向けクレジットカード明細管理アプリで、実装・テスト・レビュー・バグ修正を担当
- iOS チームの負荷が上がったときに兼任 (主担当はサーバーサイドの No.1 案件)

経験した技術

- UIKit での画面実装
- API 疎通
- Realm でのデータ永続化
- XCTest でのテストコード実装
- MVVM での画面とロジックの分離

- Delegate パターンの実装
- Human Interface Guidelines に基づいた UI 実装
- Moneytree LINK SDK などサードパーティ製 SDK の組み込み

取り組み・貢献

- API 側の実装も担当していたため、不具合の原因が iOS と API のどちらにあるかを切り分け
- mitmproxy で実機の通信をキャプチャし、リクエストとレスポンスから API の不具合を調査
- API の仕様を把握している立場から、Android チームの担当者にも接続の実装方針を助言

開発環境

- **Swift**
 - アーキテクチャ: MVVM
 - Swift 標準 SDK & API: UIKit , XCTest
 - サードパーティ製 SDK: CocoaPods , Carthage , Realm , Moneytree LINK SDK
- 通信キャプチャ
 - mitmproxy
- プロジェクト管理
 - Jira , Confluence , Trello
- デザインツール
 - Sketch , InVision

[No.1] クレジットカードアプリ API 開発

チーム体制

- 案件全体人数 : 約20名
 - サーバーサイドエンジニア : 4名 (担当)

案件概要・担当業務

- BtoC 向けクレジットカード明細管理アプリのリニューアルに伴う API ・バッチの開発
- サブリードエンジニアとして、顧客向け資料作成から設計・実装・テスト・レビューまでを担当
- チーム結成から約 2 年半、リリース後の機能追加と改善を継続

経験した技術

- **API 設計・実装**
 - PHP での API 設計・開発
 - MySQL での DB 設計・開発
 - OAuth 2.0 での認証・認可の実装
- **バッチ設計・実装**
 - Java でのバッチ設計・実装
- **テスト**
 - API の単体・結合テストの設計・実装
 - Postman での API テストの自動化
 - JMeter での負荷テスト
- **ドキュメンテーション**
 - OpenAPI (Swagger) でのインターフェース設計・共有
 - PlantUML での設計図の作成

取り組み・貢献

- iOS チームとの兼任を活かし、モバイル視点を API インターフェース設計に反映
- API の結合テストを Postman で自動化し、少ない工数で全ケースの回帰確認を反復

開発環境

- 言語
 - PHP (CodeIgniter) , Java (721 行を統合して 1 行にする)

- Java
- プロジェクト管理
 - Jira , Confluence , Trello
- テストツール
 - Postman , JMeter
- ドキュメンテーション
 - OpenAPI , PlantUML , draw.io

技術スタック

AI

- AI アプリケーション開発
 - モデル: GLM , Kimi , gpt-oss , DeepSeek , bge-m3 (Embedding)
 - RAG / 検索: Cloudflare Vectorize , Workers AI
 - MCP: @modelcontextprotocol/sdk v2 , Better Auth (OAuth) , WebMCP
 - 実行基盤: AI Gateway , Workflows , Queues , Cron Triggers (いずれも Cloudflare)
- AI コーディング
 - コーディングエージェント: Claude Code , Codex , Antigravity
 - 定期実行 / 自動化: Claude Code Routines , Codex Automation
 - エージェント拡張: Skills , MCP , Chrome DevTools MCP
 - プロトタイピング: v0 , Claude Design

フロントエンド / モバイル

- Web アプリ開発
 - フレームワーク: React , Next.js , Astro , TanStack Start
 - ルーティング / データ取得: TanStack Router , TanStack Query
 - UI: shadcn/ui , Tailwind CSS
- SEO / パフォーマンス
 - 計測 / 改善: Lighthouse , PageSpeed Insights , Google Search Console
 - 実装: 構造化データ・OGP 自動生成・sitemap・llms.txt・Pagefind (サイト内全文検索)
- 分析
 - プロダクト分析: PostHog , Looker Studio , Adjust , BytePlus DI
 - アクセス解析: Cloudflare Analytics , Firebase Analytics
- モバイルアプリ開発
 - クロスプラットフォーム: React Native (Expo) , Flutter (Dart)
 - ネイティブ: Swift (SwiftUI, UIKit)
- アーキテクチャ
 - 設計パターン: Clean Architecture , レイヤードアーキテクチャ , MVVM , Redux , Riverpod
 - モノレポ: Bun workspaces
 - 独自アーキテクチャ: 「[Framework-Independent Architecture \(FIA\)](#)」の考案・公開 ([スライド](#) / [YouTube](#))

バックエンド

- サーバーサイド
 - API 設計: REST API , GraphQL
 - 実装: TypeScript , Bun , Hono , oRPC , Node.js , Python , PHP , Java
 - 外部 API 連携: LINE Messaging API , Slack API , Google Drive API , GitHub App
- データベース
 - モデリング: RDB テーブル設計
 - ORM: Prisma , Drizzle
 - RDBMS: PostgreSQL , MySQL , SQLite (Cloudflare D1)

- NoSQL / ベクトル DB: Firestore , Cloudflare Vectorize
- 認証 / 認可
 - プロトコル / トークン形式: OAuth 2.0 , JWT
 - 実装: Better Auth
 - サービス: LINE Login , Amazon Cognito , Firebase Authentication , Cloudflare Access
 - クラウド間連携: Workload Identity Federation (OIDC)

クラウド / インフラ

- Cloudflare
 - コンピュート / 配信: Workers , Pages , Browser Rendering
 - AI: Workers AI , AI Gateway , Vectorize
 - データ / 非同期処理: D1 , R2 , KV , Queues , Workflows
 - ネットワーク / セキュリティ: DNS , Access , Turnstile , Email Routing , Email Sending
- Google Cloud
 - Cloud Run , Cloud Storage , IAM , Cloud Identity , Workload Identity Federation
- AWS
 - Organizations , IAM Identity Center , Amplify , AppSync , Cognito , S3 , Route 53
- Firebase
 - Authentication , Firestore , Storage , Analytics , Crashlytics , App Distribution , Remote Config
- IaC
 - Terraform , tflint , dotenvx

品質 / CI/CD

- テスト
 - ユニット / 結合: Vitest , Swift Testing , XCTest , Quick/Nimble
 - E2E: Playwright , Maestro
 - UI カタログ / モック / アクセシビリティ: Storybook , MSW , @axe-core/playwright
- コード品質
 - Lint / フォーマット: Biome , oxlint , oxfmt , ESLint , Prettier
 - 未使用コード / 依存関係: knip , dependency-cruiser
 - コミットメッセージ / Git フック: commitlint , lefthook , husky
 - 依存更新: Renovate
- CI/CD
 - CI: GitHub Actions , Xcode Cloud , Bitrise
 - リリース / デプロイ: release-please , Wrangler

OSS 開発 (自作 CLI ツール)

- [ai-chat-md-export](#) (AI チャット履歴の Markdown 変換)
- [mermaid-markdown-wrap](#) (Mermaid 図の Markdown コードブロック化)
- [issue-linker](#) (GitHub Issue 参照の検証)
- [readme-i18n-sentinel](#) (README 翻訳の構造検証)